Name: Simran Mandal
Roll: UG/04/BTCSE/2025/059

14. (8) A cycle

15. (b) P = (struct node*) malloc(sizeof(struct node));

16.

17. (c) The first node.

18. (b) It does not support random (indexed) access.

19.

```
P = head
while (P -> next != NULL)
{ P = P -> next }
```

20.

21. (c) The last node

22. (b) Nodes may be stored anywhere in memory.

23.
```
struct node{
    int data;
    struct node * next;}
```

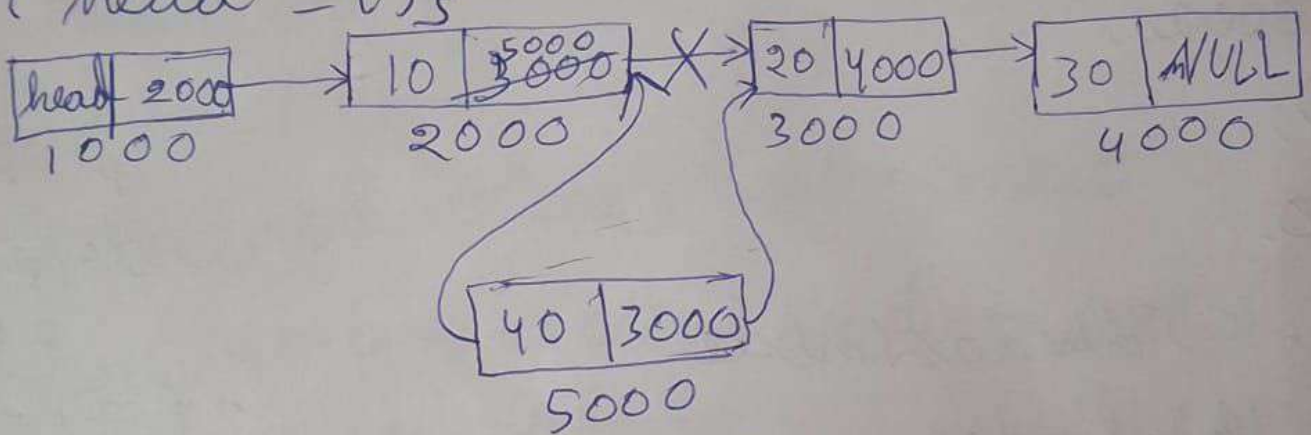
24. (10+20) + 30 = 30 + 30 = 60 + 40 = 100

25.
```
void insertEnd(struct node * head, int x)
{ struct node* t = (struct node*) malloc(
                    sizeof(struct node))
t -> data = x;
t -> next = NULL;
struct node * p = head;
```

Name:- Simran Mandal
Roll:- UU/04/BTCSE/2025/059

if (P != NULL)
while (P->next != NULL)
~~{ if (P != NULL)~~
{ P = P->next;
P->next = t; } }
else { head = t; }

27.



28. 5 → 8 → 12 → 3 → NULL

(i) struct node {
int data;
struct node *next; }
~~struct node *ptr~~
void insert (~~int value~~)
{ ~~int value = 7;~~
struct node * ~~temp~~ newnode;
newnode = (struct node*) malloc (sizeof (~~struct~~ node));
newnode -> data = ~~value~~ 7;
newnode -> next = head -> next;
head = newnode; }

7 → 5 → 8 → 12 → 3 → NULL

Data Structures & Algoritham

Name :- Simran Mandal
Roll :- UG/04/BTCSE/2025/059

1. Each node of a singly linked list contains:
(b) Data and a pointer to the next node

2. (c) NULL

3. (b) Self-referential structure

4. (b) head == NULL

5. ~~(a) newnode = head; head -> next~~

5. (d) newnode -> next = head -> next;
head = newnode.

6. (b) temp = head; head = head -> next;
free(temp);

7. (C) Traversing from tail to head.

8. (a) n-1;

9. (c) copying the data of p -> next into p, then deleting p -> next

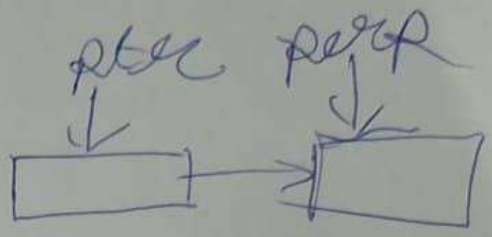
10. (a) while (p -> next != NULL)

11. (b) p -> next == NULL

12.

13. (b) Two

ii) voiddel(struct node*head)



{ struct node * perR, * ptr;
if(head!
head = ptr;
ptr → next = perR;
per

Delete the node containig

7 → 5 → 8 → 3 → NULL

iii) void insertend( ) [7 → 5 → 8 → 3 → 7 → NUL

{ struct node * newnode, temp
newnode = (struct node *) malloc(size of
(struct node));

newnode → data = 9;
newnode → next = NUL

temp = head;
if (temp → next != null)

{ temp = head;

if (temp = NULL)

{ while (temp → next != NULL)

{ temp = temp → next;
temp → next → newnode; }

else { head = newnode; }